Multi-Scale Detection of Anomalous Spatio-Temporal Trajectories in Evolving Trajectory Datasets

Chenhao Wang
University of Electronic Science and Technology of China
Chengdu, China
chwang@std.uestc.edu.cn

Lisi Chen
University of Electronic Science and Technology of China
Chengdu, China
lchen012@e.ntu.edu.sg

Shuo Shang*
University of Electronic Science and Technology of China
Chengdu, China
jedi.shang@gmail.com

Christian S. Jensen
Aalborg University
Aalborg, Denmark
csj@cs.aau.dk

Panos Kalnis
King Abdullah University of Science and Technology
Thuwal, Saudi Arabia
panos.kalnis@kaust.edu.sa

ABSTRACT

A trajectory is a sequence of timestamped point locations that captures the movement of an object such as a vehicle. Such trajectories encode complex spatial and temporal patterns and provide rich information about object mobility and the underlying infrastructures, typically road networks, within which the movements occur. A trajectory dataset is evolving when new trajectories are included continuously. The ability to detect anomalous trajectories in online fashion in this setting is fundamental and challenging functionality that has many applications, e.g., location-based services. State-of-the-art solutions determine anomalies based on the shapes or routes of trajectories, ignoring potential anomalies caused by different sampling rates or time offsets. We propose a multi-scale model, termed MST-OATD, for anomalous streaming trajectory detection that considers both the spatial and temporal aspects of trajectories. The model's multi-scale capabilities aim to enable extraction of trajectory features at multiple scales. In addition, to improve model evolvability and to contend with changes in trajectory patterns, the model is equipped with a learned ranking model that updates the training set as new trajectories are included. Experiments on real datasets offer evidence that the model can outperform state-of-the-art solutions and is capable of real-time anomaly detection. Further, the learned ranking model achieves promising results when updating the training set with newly arrived trajectories.

CCS CONCEPTS

• **Information systems** → **Spatial-temporal systems**; • **Computing methodologies** → **Artificial intelligence**.

*Corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
KDD '24, August 25–29, 2024, Barcelona, Spain
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0490-1/24/08
https://doi.org/10.1145/3637528.3671874

KEYWORDS

Anomalous trajectory detection, Spatio-temporal, Evolving datasets

ACM Reference Format:
Chenhao Wang, Lisi Chen, Shuo Shang, Christian S. Jensen, and Panos Kalnis. 2024. Multi-Scale Detection of Anomalous Spatio-Temporal Trajectories in Evolving Trajectory Datasets. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24), August 25–29, 2024, Barcelona, Spain.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3637528.3671874

1 INTRODUCTION

With the diffusion of mobile computing and geo-positioning technologies, spatial trajectories of moving objects (e.g., vehicles or pedestrians) are being collected at an unprecedented scale [23, 33]. Such trajectories, in the form of sequences of timestamped locations, capture rich movement information, and they fuel a range of urban computing applications, such as movement pattern mining [31, 32], traffic flow control [19, 30], local-event exploration [9, 26], and other urban computing applications.

Given rich movement information, anomalous trajectory detection has important applications in location-based services. For example, anomalous trajectories can reveal abnormal driving behavior. In online taxi platforms, detecting such behavior can help prevent passengers from being overcharged due to unnecessary detours [3, 15]. As another example, anomalous trajectory detection can help detect and predict events such as accidents and congestion in road networks [18, 27], thus providing technical support and foundation for location-based service providers such as online navigation and intelligent maps. Anomalous trajectory detection refers to the process of identifying trajectories that exhibit significant deviations from normal trajectories. A trajectory is considered to be anomalous if it does not follow a normal route [24, 29]. Consider the example in Figure 1, where we have a set of trajectories between a source and a destination. For the purpose of the example, we assume that trajectory T_1 (red) is a normal trajectory, while trajectory T_2 (blue) is anomalous as its route deviates significantly from that of T_1.

While the detection of anomalous trajectories has attracted substantial attention, two key challenges still need to be addressed:

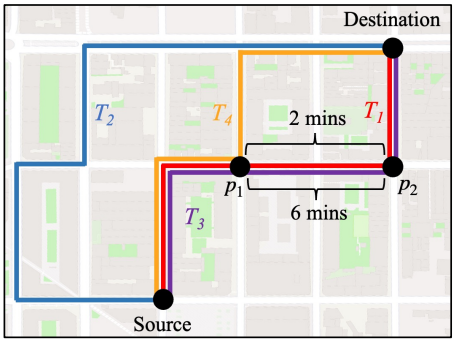


Figure 1: Illustration of spatio-temporal anomalies. T_1, T_2, T_3, and T_4 are trajectories from the same source to the same destination.

(1) **Inability to detect spatio-temporal anomalies.** Most existing detection methods [2, 13, 15, 16, 20, 24, 28, 29] focus primarily on detecting spatially anomalous trajectories. These methods consider a trajectory to be anomalous if it deviates from the route taken by most other trajectories. These methods do not take into account temporal differences between trajectories that follow the same route. However, being able to detect temporal anomalies also has important uses and should not be ignored. Consider Figure 1. Trajectory T_3 (purple) follows the same route as T_1 (red). However, T_3 takes considerably longer to travel from p_1 to p_2 than T_1 , i.e., 6 minutes vs. 2 minutes. Assuming that T_1 is normal, T_3 should then also be considered as an anomalous trajectory. In real-world applications, ensuring consistent sampling time intervals is challenging due to different types of sampling devices and missing data [25]. Although a trajectory is considered as a normal trajectory in terms of its location coordinates, the time intervals between consecutive points may differ significantly from what is seen in a normal trajectory. Thus, the trajectory is still anomalous. Differences in sampling intervals and missing sampling points can result in an anomalous trajectory being indistinguishable from a normal trajectory. It is therefore important to consider temporal differences when detecting anomalous trajectories.

(2) **No support for updates.** As machine learning continues to advance, many recent approaches to trajectory anomaly detection employ machine learning to leverage these advances. Machine learning uses trained models to extract features from trajectories that can be used to detect anomalies [7, 15, 20]. However, currently available learning-based methods focus on detecting trajectories using pre-trained weights and do not provide support for updating models as new data is collected. Irreversible changes to a road network such as new roads and traffic regulations, e.g., one-way lanes, speed limits, and traffic lights, call for updatable trajectory sets and models. Therefore, it is essential to be able to update models as new trajectories become available. To achieve high trajectory anomaly detection performance, it is necessary to achieve efficiently updatable training that keeps models up-to-date with the latest data, while avoiding costly retraining of entire models from scratch.

In this light, we propose Multi-scale Spatio-temporal Online Anomalous Trajectory Detection (MST-OATD) method. In MST-OATD, a trajectory is represented by a spatio-temporal embedding sequence. A multi-scale inference strategy is proposed to capture multi-scale spatio-temporal trajectory features. We partition trajectories into segments of different sizes. The multi-scale strategy then combines spatial and temporal features of trajectory segments at different scales. When inferring normal route features, we used a Gaussian mixture model to model features, where different Gaussian components can represent different types of trajectories. In addition, inspired by learned indexing [12], a learned ranking model is introduced to learn the ranks of the probability of a trajectory following a particular type. This ranking model can obtain ranks with constant complexity as new trajectories are collected, and the ranks are used to update the training trajectory set when new trajectories are collected.

Our contributions are summarized as follows. First, we propose a novel anomalous trajectory detection framework called MST-OATD that can detect both spatial and temporal trajectory anomalies. Second, we develop a strategy to capture trajectory features at multiple scales that is effective and efficient. The relationships between trajectory segments at different scales are used for latent route inference and generation. Third, we introduce a learned ranking model to learn the rank of a trajectory based on the probability of following each Gaussian component. This is used to update the training set at constant complexity. Finally, we report on extensive experiments with online anomalous trajectory detection on two large-scale real-world datasets, finding that MST-OATD outperforms existing methods. We also find that using the ranking-based update strategy enables better detection results than other update strategies. Our code is available at https://github.com/chwang0721/MST-OATD.

2 RELATED WORK

2.1 Anomalous Trajectory Detection

Existing methods for identifying spatially anomalous in trajectories can be categorized as either metric-based or learning-based methods [15]. The former typically use hand-crafted features and predefined metrics such as distance or density to detect anomalies. An early study [6] identifies typical trajectories between locations of interest and computes the distance of each segment of the target trajectory to reference trajectories. IBAT [29] detects anomalies by finding the trajectories with the same SD pair as the reference trajectory and employs the notion of degree of isolation. IBOAT [2] is another isolation-based method for online detection. Learning-based methods learn the features of normal routes and use these to calculate anomaly scores of target trajectories. The probabilistic method DB-TOD [24] models driving behavior/preferences from historical trajectories using reinforcement learning. The RNN-based method ATD-RNN [20] characterizes a trajectory by learning its trajectory embedding using a stacked RNN. Recently, the generative sequence model GM-VSAE [15] has been proposed. Inspired by the VAE [10], it uses a Gaussian mixture model to represent trajectory features in latent space. ATROM [5] is a probabilistic metric learning model that focuses on recognizing which type of anomaly a trajectory exhibits. SafeTrip [21] uses several factors such as distance and speed to estimate the probability of abnormal safety events in a ride-hailing scenario.

Compared to detecting spatially anomalous trajectories, few methods focus on the temporal information in trajectories. TPRO [34]

develops a time-dependent graph to find popular routes in a given time period as reference routes. An improved version of TPRO, TPRRO [35] is proposed to speed up the detection process. The recent time-dependent method DeepTEA [7] uses traffic patterns as a reference and combines these with trajectory patterns. However, these methods are time-dependent methods, they focus on obtaining normal routes at different time periods, which is different from our goal of detecting spatio-temporal anomalies.

2.2 Rank Prediction Model

The B-tree and its variants (e.g, B+-tree [1]) are the most commonly used index structures for querying. Given a query, they aim to return the position of a value within a key-sorted set. A recent study of learned indexing [12] argues that the B-tree can be thought of as a learnable model. The position of a key within a sorted array can be approximated as a cumulative distribution function. As a result, the model can be implemented using machine learning, such as linear regression. Inspired by this, a similarity search called LIMS [22] uses a polynomial function to predict the ranks of elements in clusters. In our study, this technique can be used to predict the ranks of trajectories. Although rank prediction using this method is not exact due to the uncertainty of machine learning [4], the need for an exact lookup is not necessary in our training set update. Our focus is on updating distributions rather than performing a precise search. Therefore, the learned index technique is suitable for rank prediction in our application scenario.

3 PROBLEM FORMULATION

We proceed to present definitions and formally state the online anomalous trajectory detection problem.

3.1 Preliminaries

Trajectory Point. A trajectory point $p = (l, t)$ captures the spatial location l of a mobile object at a specific time t.

Trajectory. A trajectory $T = \langle p_1, p_2, ..., p_n \rangle$ is a sequence of trajectory points p_i that capture the movement of an object over time. Given a trajectory T, $T^{(s)}$ and $T^{(t)}$ denote its spatial and temporal aspects, respectively: $T^{(s)} = \langle l_1, l_2, ..., l_n \rangle$ and $T^{(t)} = \langle t_1, t_2, ..., t_n \rangle$.

Trajectory Segment. A trajectory segment $T[i; j]$ is a consecutive sub-sequence of trajectory $T = \langle p_1, p_2, ..., p_n \rangle$ from p_i to p_j, $1 \leq i \leq j \leq n$.

Route Type. Given a set of trajectories $\{T_1, T_2, ..., T_N\}$ from different regions of a city (e.g., downtown or suburbs) and using different types of roads (e.g., freeways or streets), we use this information to assign a route type c to each trajectory.

Trajectory Rank. The rank $rank_c(T)$ of a trajectory T is defined as the rank of the probability among all the trajectories that a trajectory T has, or follows, route type c. In the example in Figure 1, we assume that T_1 (red) represents a trajectory following the route type c. Then we assume that the probabilities of T_2 (blue), T_3 (purple), and T_4 (orange) following route type c are 0, 1, and 0.5, respectively. Therefore, $rank_c(T_3) = 1$, $rank_c(T_4) = 2$, and $rank_c(T_2) = 3$. The probabilities are calculated based on a Gaussian mixture model, to be described in Section 4.2.2.

3.2 Problem Statement

Online Anomalous Trajectory Detection. A trajectory T is considered anomalous if its route deviates significantly from the normal routes or its time consumption differs significantly from those of normal trajectories with the same route. The objective of online anomalous trajectory detection is to acquire knowledge of regular route patterns and update the anomaly score at each time point. Section 5 outlines the methodology for calculating the anomaly score $Score(T)$. The range of the score is $[0, 1]$, and $Score(T)$ can reflect the combination of spatial and temporal anomalies of T.

4 THE MST-OATD MODEL

We proceed to present the MST-OATD method. The overall architecture of MST-OATD is illustrated in Figure 2.

4.1 Trajectory Embedding Computation

To extract trajectory features, it is essential to represent trajectories using low-dimensional embeddings. To achieve this, we compute spatial and temporal embeddings and perform multi-scale fusion.

4.1.1 Spatial Embedding. We map spatial locations of trajectories to cells in a uniform grid following a previous study [14]. Given a trajectory T, the spatial sequence $T^{(s)} = \langle l_1, l_2, ..., l_n \rangle$ is transformed into sequence $\mathbf{g} = \langle g_1, g_2, ..., g_n \rangle$, where g_i denotes a grid cell. We use e_{g_i} to represent the grid cell id embedding of g_i. However, grid cell id embedding is not a complete representation of a trajectory point because it ignores the positional relationships between the grid cells. Inspired by Graph Convolutional Networks [11], we generate an aggregation matrix H for the grid cells:

$$H = \widetilde{D}^{-\frac{1}{2}} W \widetilde{A} \widetilde{D}^{-\frac{1}{2}}, \tag{1}$$

where A is the adjacency matrix. In the grid mapping, adjacent grid cells are cells with common edges and $\widetilde{A} = A + I$, $\widetilde{A}$ is used to aggregate features of a grid cell and its adjacent grid cells. Next, degree matrix $\widetilde{D}$ is calculated as $\widetilde{D}_{ii} = \sum_j \widetilde{A}_{ij}$, and W is a weight matrix for $\widetilde{A}$, which is learned to obtain the weights of adjacent grid cells since the influence of adjacent grid cells vary. For instance, cells connected by a road, influence each other more than cells without a road connection.

Therefore, the spatial embedding $e_i^{(s)}$ can be calculated by multiplying the aggregation matrix and grid cell id embedding:

$$e_i^{(s)} = H e_{g_i} = \widetilde{D}^{-\frac{1}{2}} W \widetilde{A} \widetilde{D}^{-\frac{1}{2}} e_{g_i} \tag{2}$$

Based on the method above, the spatial trajectory $T^{(s)}$ can be embedded as a sequence of vectors $\mathbf{e}^{(s)} = \langle e_1^{(s)}, e_2^{(s)}, ..., e_n^{(s)} \rangle$.

4.1.2 Temporal Embedding. Compared to the spatial embedding, the temporal embedding is more difficult to compute because the method used for spatial embedding may fail to capture periodic features, which is an important characteristic of temporal information. Instead, inspired by Time2Vec [8], we learn temporal embeddings using a neural network with a periodic activation function to extract periodic and non-periodic features. For each timestamp t in a trajectory, we represent t by a tuple τ consisting of values for different levels of time units (e.g., dates, hours, minutes and seconds).

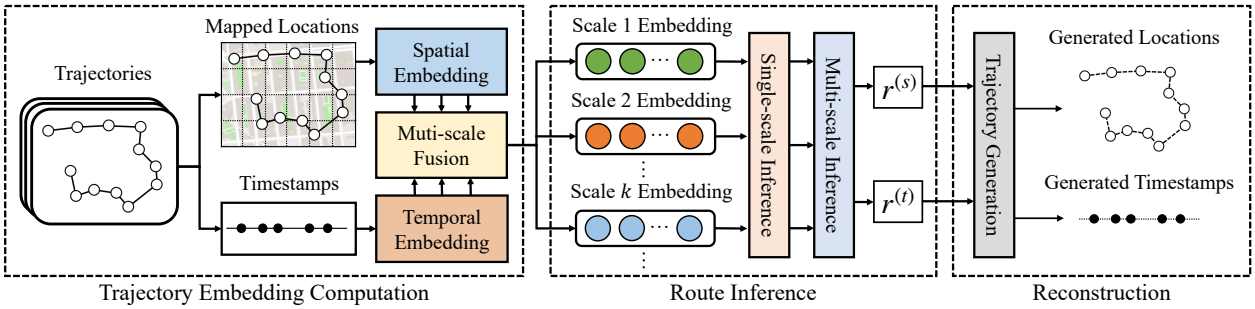


Figure 2: An overview of MST-OATD. It comprises three stages: trajectory embedding computation, route inference, and trajectory reconstruction.

The temporal embedding $e^{(t)}$ is given in Equation 3:

$$e^{(t)} = [f_1(\tau) \| \mathcal{F}(f_2(\tau))], \tag{3}$$

where $f_1(\cdot)$ and $f_2(\cdot)$ are learnable fully connected layers, $\mathcal{F}$ denotes a periodic activation function that helps capture periodic behaviors at different levels without the need for feature engineering, $[\cdot\|\cdot]$ is the concatenation operator. In MST-OATD, we choose the *sine* function as $\mathcal{F}$, representing periodicity at different time units. With Equation 3, the temporal information of a trajectory $T^{(t)} = \langle t_1, t_2, ..., t_n \rangle$ can be embedded into a sequence of vectors $\mathbf{e}^{(t)} = \langle e_1^{(t)}, e_2^{(t)}, ..., e_n^{(t)} \rangle$.

4.1.3 Multi-scale Fusion. A trajectory is a sequence of trajectory points, and the connections between different segments of a trajectory play a crucial role in representing the trajectory's features. As single-scale methods focus mainly on individual points within a trajectory, they face challenges in achieving a comprehensive representation of an entire trajectory. Furthermore, it is essential to integrate spatial and temporal information. To address these issues, we provide a multi-scale fusion module that effectively captures spatio-temporal features across multiple scales.

Specifically, a cross-attention module is adopted to fuse spatial and temporal embeddings. The spatial and temporal attention results $\hat{\mathbf{e}}^{(s)}$ and $\hat{\mathbf{e}}^{(t)}$ are computed as follows:

$$\begin{aligned} \hat{\mathbf{e}}^{(s)} &= \text{Softmax}(\mathbf{e}^{(t)} W_q^{(s)} \mathbf{e}^{(s)} {W_k^{(s)}}^T / \sqrt{d_k}) \mathbf{e}^{(s)} W_V^{(s)} \\ \hat{\mathbf{e}}^{(t)} &= \text{Softmax}(\mathbf{e}^{(s)} W_q^{(t)} \mathbf{e}^{(t)} {W_k^{(t)}}^T / \sqrt{d_k}) \mathbf{e}^{(t)} W_V^{(t)}, \end{aligned} \tag{4}$$

where $W_q^{(s)}$, $W_k^{(s)}$, and $W_V^{(s)}$ are the weights for spatial cross-attention; $W_q^{(t)}$, $W_k^{(t)}$, and $W_V^{(t)}$ are the weights for temporal cross-attention; and d_k is the dimensionality of $W_k^{(s)}$ and $W_k^{(t)}$. The trajectory embedding $\mathbf{e}$ is then calculated as shown in Equation 5:

$$\mathbf{e} = [(\text{Norm}(FFN(\hat{\mathbf{e}}^{(s)}) + \mathbf{e}^{(s)}) \| \text{Norm}(FFN(\hat{\mathbf{e}}^{(t)}) + \mathbf{e}^{(t)}))] \tag{5}$$

Here, *FFN* denotes a feed-forward network, and Norm is the Layer Normalization operation. We use $\mathbf{e}_k$ to represent a trajectory embedding at scale k. Specifically, since we have a sequence of embeddings $\mathbf{e} = \langle e_1, e_2, ..., e_n \rangle$ of length n, $\mathbf{e}_k$ is computed as follows:

$$\mathbf{e}_k = \langle \hat{e}_{k1}, \hat{e}_{k2}, ..., \hat{e}_{k\lceil n/s_k \rceil} \rangle, \tag{6}$$

where s_k denotes the segment size of scale k, $\hat{e}_{ki}$ is the mean of the values of embeddings in the i-th segment of scale k, $\lceil \cdot \rceil$ denotes the rounding-up operation, $\hat{e}_{k\lceil n/s_k \rceil}$ is the last segment of trajectory T because the length of a trajectory is not necessarily divisible by the size of the segment, which are calculated as shown in next:

$$\hat{e}_{ki} = \begin{cases} \sum_{j=(i-1)s_k+1}^{i \times s_k + 1} e_j / s_k & i < \lceil n/s_k \rceil \\ \sum_{j=(i-1)s_k+1}^{n} e_j / s_k & i = \lceil n/s_k \rceil \end{cases} \tag{7}$$

In order to capture the correlation between the segments in a trajectory, we use an attention layer for embeddings at different scales, the aim being to compute the attention score of segments in the same trajectory. The calculation is shown in Equation 8:

$$\tilde{e}_{ki} = \sum_j \text{Softmax}(\alpha_{i,j}) \hat{e}_{kj}, \tag{8}$$

where $\alpha_{i,j} = W_\alpha \cdot \tanh(W_i \cdot \hat{e}_{ki} + W_j \cdot \hat{e}_{kj})$; W_α, W_i, and W_k are the parameters to learn; and $\tilde{e}_{ki}$ is the i-th segment embedding at scale k after the attention layer.

We use $\tilde{\mathbf{e}_k}$ to represent the embedding of trajectory T at scale k after multi-scale fusion, and $\tilde{\mathbf{e}_k}$ is calculated using the method above. Different scales capture different characteristics of trajectories. For example, smaller scales can capture intricate details such as corners. Conversely, larger scales are required to capture trajectory features such as their overall shape.

4.2 Route Inference

To extract spatio-temporal features of the trajectory T at multiple scales, we propose an inference network $q_\theta(r^{(s)}, r^{(t)}|T)$ to infer latent route vectors given a trajectory T. Here, θ denotes the parameters of $q_\theta(r^{(s)}, r^{(t)}|T)$ to be learned, where $r^{(s)}$ and $r^{(t)}$ are spatial and temporal latent vectors. To cope with the uncertainty of trajectories, we model $r^{(s)}$ and $r^{(t)}$ using Gaussian distributions:

$$\begin{aligned} r_T^{(s)} &\sim q_\theta(r^{(s)}|T) = \mathcal{N}(\mu^{(s)}, {\sigma^2}^{(s)}) \\ r_T^{(t)} &\sim q_\theta(r^{(t)}|T) = \mathcal{N}(\mu^{(t)}, {\sigma^2}^{(t)}), \end{aligned} \tag{9}$$

where $\mu^{(s)}$, ${\sigma^2}^{(s)}$, $\mu^{(t)}$, and ${\sigma^2}^{(t)}$ are parameters of the spatial and temporal Gaussian distributions.

The route inference stage consists of single-scale and multi-scale inference. Single-scale inference acquires latent vectors at a particular scale, while multi-scale inference combines vectors at different scales.

4.2.1 Single-scale Inference. We provide a specialized module for route inference, focusing on a specific scale. Our approach employs recurrent neural networks (RNN) to enable single-scale inference. The RNNs operate at each timestamp t_i, taking in the trajectory embedding $\tilde{e}_i$ and generating a hidden state h_i as follows:

$$h_i = g_1(\tilde{e}_i, h_{i-1}), \quad i = 1, 2, ..., n \tag{10}$$

Here, $g_1(\cdot)$ represents the RNN module, while h_{i-1} denotes the $i-1$-th hidden state. Using this method, $h_i^{(s)}$ and $h_i^{(t)}$ can be obtained given $\tilde{e}_i$, $h_{i-1}^{(s)}$, and $h_{i-1}^{(t)}$. We use $h^{(s)}$ and $h^{(t)}$ to refer to the final hidden states derived from the spatial and temporal embeddings, respectively.

4.2.2 Multi-scale Inference. To combine single-scale trajectory features, we propose a probability-based multi-scale inference module for latent route inference. Gaussian mixture models are used to model the spatial and temporal latent vectors to contend with the complexity of the trajectories. The parameters of the spatial and temporal Gaussian mixture distributions at scale k can be obtained by performing linear transformations on ${h^{(s)}}_k$ and ${h^{(t)}}_k$:

$$\begin{aligned} \mu^{(s)} &= f_3(\textstyle\sum_k \lambda_k^{(s)} {h^{(s)}}_k), \quad \sigma^{2(s)} = f_4(\textstyle\sum_k \lambda_k^{(s)} {h^{(s)}}_k) \\ \mu^{(t)} &= f_5(\textstyle\sum_k \lambda_k^{(t)} {h^{(t)}}_k), \quad \sigma^{2(t)} = f_6(\textstyle\sum_k \lambda_k^{(t)} {h^{(t)}}_k), \end{aligned} \tag{11}$$

where ${h^{(s)}}_k$ and ${h^{(t)}}_k$ are the final hidden states of the spatial and the temporal embeddings at scale k, $\lambda_k^{(s)}$ and $\lambda_k^{(t)}$ are spatial and temporal parameters at scale k, and $f_3(\cdot)$, $f_4(\cdot)$, $f_5(\cdot)$, and $f_6(\cdot)$ are fully connected layers. Further, $\mu^{(s)}$, $\sigma^{2(s)}$, $\mu^{(t)}$, and $\sigma^{2(t)}$ are the multi-scale processed parameters of spatial and temporal Gaussian mixture distributions. These two distributions contain information about spatially and temporally normal routes.

Furthermore, since vehicles may travel in different regions of a city (e.g., downtown or suburbs) and on different types of roads (e.g., freeways or streets), it is not reasonable to model all trajectories with a single distribution. Similarly, normal trajectories are different at different times, e.g., morning and evening rush hour trajectories differ from ones at other times, so it also makes sense to temporally categorize trajectories into different types. Motivated by this, we assume that there are C types of routes both spatially and temporally. We use a multinomial distribution $p_\phi(c) = Mult(\pi)$ to represent the distribution of route types, where c is a variable that determines the type of a trajectory and π is the distribution parameter. For each type, a Gaussian distribution is used to model the latent vector, as shown in Equations 12:

$$p_\phi(r^{(s)}|c^{(s)}) = \mathcal{N}(\mu^{(s)}_{c^{(s)}}, \sigma^{2(s)}_{c^{(s)}}), p_\phi(r^{(t)}|c^{(t)}) = \mathcal{N}(\mu^{(t)}_{c^{(t)}}, \sigma^{2(t)}_{c^{(t)}}) \tag{12}$$

Here, $p_\phi(r|c)$ denotes the probability of a type c route being traversed by the trajectories, and $r^{(s)}$ and $r^{(t)}$ are the spatial and temporal latent route vectors. Next, $\mu^{(s)}_{c^{(s)}}$ and $\sigma^{2(s)}_{c^{(s)}}$ denote the mean and variance of the spatial distribution of type $c^{(s)}$, and $\mu^{(t)}_{c^{(t)}}$ and $\sigma^{2(t)}_{c^{(t)}}$ denote the parameters of the temporal distribution of type $c^{(t)}$). The latent vectors $r^{(s)}$, and $r^{(t)}$ and the route types $c^{(s)}$, and $c^{(t)}$ can be inferred from Equation 13:

$$q_\theta(r^{(s)}, r^{(t)}, c^{(s)}, c^{(t)}|T) = q_\theta(r^{(s)}, r^{(t)}|T) q_\theta(c^{(s)}, c^{(t)}|T) \tag{13}$$

Basically, $q_\theta(c^{(s)}, c^{(t)}|T)$ can be considered as the probability of spatial type $c^{(s)}$ and temporal type $c^{(t)}$ given the route of a trajectory T. Since the trajectory T can be encoded into spatial and temporal latent vectors $r_T^{(s)}$ and $r_T^{(t)}$, $q_\theta(c^{(s)}, c^{(t)}|T)$ can also be written as $p_\phi(c^{(s)}, c^{(t)}|r_T^{(s)}, r_T^{(t)})$. Then, $p_\phi(c|r_T)$ can be converted by the Bayesian formula as shown in Equation 14:

$$\begin{aligned} q_\theta(c^{(s)}, c^{(t)}|T) &= p_\phi(c^{(s)}, c^{(t)}|r_T^{(s)}, r_T^{(t)}) \\ &= \frac{p_\phi(c^{(s)}) p_\phi(r_T^{(s)}|c^{(s)})}{\sum_{i=1}^{C} p_\phi(i) p_\phi(r_T^{(s)}|i)} \cdot \frac{p_\phi(c^{(t)}) p_\phi(r_T^{(t)}|c^{(t)})}{\sum_{j=1}^{C} p_\phi(j) p_\phi(r_T^{(t)}|j)} \end{aligned} \tag{14}$$

Therefore, the spatial and temporal latent vectors and their types can be inferred using $p_\phi(r|c)$ and $p_\phi(c)$, which are defined above.

4.3 Trajectory Reconstruction

To enable online detection, we design a generation network $p_\gamma(T|r)$, where γ represents the parameters of the network. The distribution $p_\gamma(T|r)$ denotes the probability that a trajectory T follows normal routes contained in r. RNNs are also used to generate trajectories. Given a sequence of embeddings $\mathbf{e}_{<i} = \langle e_1, e_2, ..., e_{i-1} \rangle$ and a latent route vector r, the hidden state m_i is updated as shown in Equation 15:

$$m_i = g_2(e_i, m_{i-1}) \quad i = 1, 2, ..., n \quad m_0 = r, \tag{15}$$

where $g_2(\cdot)$ is an RNN to be learned, and m_0 is the initial hidden state, which is also the spatial latent route vector. The e_i at the i-th step is generated according to Equation 16:

$$e_i \sim p_\gamma(e_i|\mathbf{e}_{<i}, r) = p_\gamma(e_i|m_{i-1}). \tag{16}$$

With this method, $e_i^{(s)}$ can be calculated given a sequence of spatial embeddings ${\mathbf{e}^{(s)}}_{<i}$ and spatial latent route vector $r^{(s)}$; and $e_i^{(t)}$ can be calculated given a sequence of temporal embeddings ${\mathbf{e}^{(t)}}_{<i}$ and temporal latent route vector $r^{(t)}$.

Using the generation network, the spatial and temporal aspects of trajectory T, $T^{(s)}$ and $T^{(t)}$, can be generated sequentially by $p_\gamma(T^{(s)}|r^{(s)})$ and $p_\gamma(T^{(t)}|r^{(t)})$.

4.4 Optimization

The parameters to be learned include the spatial and temporal embeddings, θ for route inference, and γ for trajectory generation. The training objective is to maximize the marginal log-likelihood of generated trajectories:

$$\log p_\gamma(T_1, T_2, ..., T_N) = \sum_{i=1}^{N} \log p_\gamma(T_i^{(s)}) p_\gamma(T_i^{(t)}), \tag{17}$$

where N is the number of trajectories used for training. The marginal log-likelihood is optimized by maximizing the lower bound on the sum of marginal likelihoods $\mathcal{L}(\theta, \phi, \gamma; T^{(s)})$ and $\mathcal{L}(\theta, \phi, \gamma; T^{(t)})$. Appendix provides the details on calculating the lower bound.

5 ONLINE ANOMALOUS TRAJECTORY DETECTION BY MST-OATD

To detect anomalous trajectories, we use the detect-by-generation strategy. Given a trajectory T and latent route vectors $r^{(s)}$ and $r^{(t)}$, the generative networks $p_\gamma(T^{(s)}|r^{(s)})$ and $p_\gamma(T^{(t)}|r^{(t)})$ are used to

compute the probability that T was generated from the latent vector r. To find the most likely route type in the spatial and temporal latent space for each trajectory, we compute the probability of the trajectory following all types, and we select the highest type as the final type. The anomaly score of a trajectory T is defined in Equation 18:

$$Score(T) = 1 - \arg\max_{c^{(s)}, c^{(t)}} \exp[\frac{\log p_\gamma(T^{(s)}|\mu^{(s)}_{c^{(s)}}) p_\gamma(T^{(t)}|\mu^{(t)}_{c^{(t)}})}{n}], \tag{18}$$

where arg max is the operation that finds the argument that maximizes the argument function, n is the length of the trajectory, which is used to normalize the anomaly score of trajectories with different lengths, and $c^{(s)}$ and $c^{(t)}$ are the types of the spatial and temporal embeddings. Here, we use $\mu^{(s)}_{c^{(s)}}$ and $\mu^{(t)}_{c^{(t)}}$ to approximate the latent vectors of each type.

For online detection, the anomaly score $Score(T_{\leq i})$ is determined by the log-likelihood at the previous timestamp and the upcoming trajectory point:

$$\begin{aligned} Score(T_{\leq i}) =& 1 - \arg\max_{c^{(s)}, c^{(t)}} \exp[\frac{\log p_\gamma(T^{(s)}_{\leq i}|\mu^{(s)}_{c^{(s)}}) p_\gamma(e^{(s)}_i|T^{(s)}_{\leq i}, \mu^{(s)}_{c^{(s)}})}{i+1} \\ &+ \frac{\log p_\gamma(T^{(t)}_{\leq i}|\mu^{(t)}_{c^{(t)}}) p_\gamma(e^{(t)}_i|T^{(t)}_{\leq i}, \mu^{(t)}_{c^{(t)}})}{i+1}] \end{aligned} \tag{19}$$

6 EVOLVING DATASET UPDATE

Sections 4 and 5 present the MST-OATD model and its application to online detection. However, as road conditions change over time and new trajectories are collected, the pre-trained spatial and temporal features may not match the current trajectory patterns. Therefore, a method to update the model with new data is needed.

A natural approach is to update the model weights with a constant number of newly collected trajectories, replacing the oldest ones. However, this approach is not always optimal. For example, if all of the newest trajectories are from a particular type, simply replacing the oldest trajectories may not result in a rapid shift in the data distribution. Our goal is to ensure that the training and target data distributions remain consistent, as this allows the model to be fine-tuned with greater efficiency. To achieve this, we propose a ranking-based update strategy that prioritizes updates of the training set based on the spatial distribution of the trajectories.

Since the update process is inherently a temporal problem, we focus on updating the training set according to the spatial distribution. More specifically, suppose we have a set of trajectories that spatially follow a Gaussian mixture distribution with C components. As discussed in Section 4.2.2, the probability that a trajectory T is of type c in the spatial domain is expressed as $p(c|T) = p(c|r^{(s)}_T)$, where $r^{(s)}_T$ is the spatial latent vector of T. We compute the probability that each trajectory follows a particular type, and then rank them in descending order. We place a trajectory in the type with the highest rank and replace the earliest trajectory in the type with the lowest rank.

Further, considering that our model supports online detection, the efficiency of updates is also critical. The input to our ranking-based model is the probability that the trajectory has a particular type, as discussed in Section 4.2.2, and the output is the rank of the probability of the trajectory in that type. Traditional search structures like B-trees can handle this problem with $O(\log n)$ time complexity. Inspired by learned indexing [12], it is possible to view the rank of a trajectory as an estimate of the cumulative distribution function (CDF). Equation 20 models the relationship between the trajectory rank $rank_c(T)$ and the probability of T having spatial type c:

$$rank_c(T) = F(p(c|r^{(s)})) * N, \tag{20}$$

where $F(\cdot)$ is the estimated cumulative distribution function and N is the total number of trajectories. Specifically, we use logistic regression to implement the model as follows:

$$rank_c(T) = [1 + e^{-(w \cdot p(c|r^{(s)}) + b)}]^{-1} * N, \tag{21}$$

where w and b are the weight and bias. The use of the Sigmoid activation function is able to restrict the output to [0, 1], which is consistent with the output range of the cumulative distribution. The regression model learns the data distribution by minimising the squared error of the function. The loss of category c is calculated as follows:

$$L(w; b) = \sum_{i=1}^{N} (y_i - rank_c(T_i))^2, \tag{22}$$

where y_i is the real rank of trajectory T_i. Using this method, the trajectory rank of T can be obtained within a constant operation. The Appendix provides the pseudo-code of evolving dataset update using the learned ranking method.

7 EXPERIMENTS

7.1 Experimental Settings

Datasets: We conducted experiments on two real-world taxi trajectory datasets, namely Porto[1] and Chengdu[2]. Porto contains an entire year of trajectories from 442 taxis in Porto, Portugal, where positions are sampled every 15 seconds. Chengdu contains over 1.4 billion trajectory points from approximately 14 thousand taxis in Chengdu, China from August 3, 2014, to August 30, 2014. In Chengdu, the sampling rate is non-uniform. In our experiments, we use a ten-day subset of the Chengdu dataset.

Prepossessing: Following a previous study [14], we converted the underlying geographical spaces of Porto and Chengdu into 100m×100m grids. In Porto, trajectories with more than 50 points are divided into trajectories with lengths between 20 and 50, and we also delete at random 30% of the points in each trajectory to make the durations between points uneven. In Chengdu, trajectories longer than 100 points are divided randomly into trajectories with lengths between 30 and 100. Table 1 gives information about the two adjusted datasets. For Porto, we use the first two months of data to evaluate the effectiveness and efficiency of MST-OATD and to train the ranking model. The remaining data is used to evaluate the performance with the evolving dataset update algorithm. Similarly, the first three days of data in Chengdu are used to evaluate the effectiveness and efficiency and to train the ranking model. The

[1] https://www.kaggle.com/c/pkdd-15-predict-taxi-service-trajectory-i/data
[2] https://js.dclab.run/v2/cmptDetail.html?id=175

remaining seven days of trajectories are used to update the training set. For each dataset, we randomly select 80% of the trajectories for training and 20% for testing.

Table 1: Dataset Statistics

Dataset	# Trajectories	# Trajectory Points	# Grid Size
Porto	1,198,551	35,274,322	[51, 119]
Chengdu	2,124,763	128,837,312	[167, 154]

Ground Truth: In the absence of a label dataset for anomaly detection, some studies attempt to label anomalies manually. However, manual labeling is time-consuming and lacks uniform standards. Instead, we generate anomalous samples according to specific rules. Following previous studies, we use a perturbation scheme to generate spatial and temporal anomalies. Specifically, we use parameters α and d, where α is the fraction of continuous outlier and d is the moving distance of outliers. In GM-VSAE [15], a spatially anomalous trajectory is generated by moving a part of a trajectory by a certain distance. However, in this method, the moved part of the trajectory is not continuous. Our method moves a continuous part of a trajectory. For example, $\alpha = 0.1; d = 3$ means that 10% of a trajectory deviates by 3 grid cells. It should also be noted that trajectory perturbations affect the timestamps of the trajectories. We assume that the speed of a trajectory is constant, so as the distance traveled increases, the time interval will change to keep the number of trajectory points constant. The proportion of anomalous samples generated is set to 5% in all experiments.

Evaluation Metric: We use Precision-Recall AUC (PR-AUC) to evaluate the performance of anomalous trajectory detection. Since the anomalous samples make up a small proportion of the datasets, the positive and negative samples are unbalanced. The goal of the evaluation of anomaly detection is to evaluate the ability to discover positive samples, for which PR-AUC is suitable.

Baselines: We consider six existing methods for online anomalous trajectory detection:

- **iBAT** [29]. This isolation-based anomalous trajectory method detects anomalous trajectories by checking how much a target trajectory can be isolated from reference trajectories.
- **SAE** [17]. The sequence autoencoder is an RNN-based sequence-to-sequence model learned by minimizing a reconstruction error. The anomaly score is also defined based on the reconstruction error.
- **VSAE** [10]. The variational sequence autoencoder is a degenerate GM-VSAE model, where the latent route distribution is a Gaussian.
- **TPRRO** [35]. This method distinguishes outliers from time-dependent popular routes using a time-dependent transfer graph.
- **GM-VSAE** [15]. This Gaussian mixture variational sequence autoencoder is an RNN-based sequence-to-sequence model. This method can handle complex trajectory distributions by considering the latent space as a Gaussian mixture model.
- **DeepTEA** [7] This is a state-of-the-art time-dependent method that combines dynamic traffic conditions and trajectory patterns to detect anomalies.

7.2 Effectiveness Evaluation

Table 2 shows experimental results on the Porto and Chengdu datasets. To evaluate the online detection capability, we vary the observed ratio ρ of the trajectories, i.e, only part of the trajectory are observed during detection. We can find that compared to the metric-based method iBAT and TPRRO, the learning-based methods have significant advantages. The proposed MST-OATD outperforms all baselines. MST-OATD has an average promotion of 18.6% on Porto and 2.9% on Chengdu compared to the best existing method, DeepTEA. The average promotion is the mean value of the PR-AUC improvements over DeepTEA divided by DeepTEA's PR-AUC in each set of experiments. For full trajectory detection, MST-OATD achieves high PR-AUC results even when the offset distance is small, which means that it is capable of outlier detection even when anomalous parts are not obvious. For example, when $\alpha = 0.2$ and $d = 2$, the PR-AUC can be as high as 0.892 and 0.949 on Porto and Chengdu, respectively. For online detection, we set the observed ratio to 0.5 and 0.7, which means that only the first 50% and 70% of a trajectory can be observed. MST-OATD can again outperform all baselines including two methods that leverage temporal information TPRRO and DeepTEA. On Chengdu, the improvements are smaller than on Porto due to the higher PR-AUC values, making improvements it more difficult. In addition, the shorter time horizon of Chengdu limits the use of temporal information.

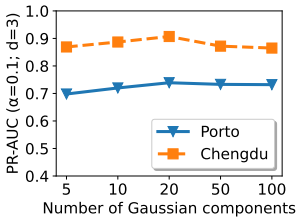


Figure 3: Effect of the number of Gaussian components.

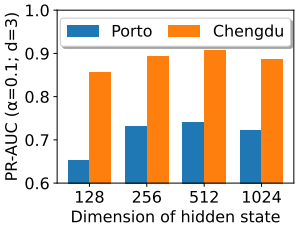


Figure 4: Effect of the dimension of hidden state.

7.3 Hyperparameter Sensitivity Analysis

We perform parameter sensitivity experiments on the two datasets and present our analysis. Two hyperparameters influence the performance of MST-OATD the most: the dimension of the RNN hidden state and the number of Gaussian components. The trajectory perturbation parameter is set to $\alpha = 0.1, d = 3$ in the ablation study, and we focus on the result of detecting complete trajectories. The number of Gaussian components varies from 5 to 100, and the dimensionality of the hidden state varies from 128 to 1024. The experimental results are shown in Figures 3 and 4. We find that the PR-AUC is highest when the number of Gaussian components is 20 on both Porto and Chengdu. The best detection performance is achieved when the dimensionality of the hidden state is set to 512 on both Porto and Chengdu.

7.4 Ablation Study

We conduct an ablation study on Porto and Chengdu to assess the effectiveness of the two modules of MST-OATD. We thus compare MST-OATD with two variants:

Table 2: Performance Comparison on Porto and Chengdu Dataset (PR-AUC)

Dataset	Porto									Chengdu								
Parameters	$\alpha = 0.1; d = 3$			$\alpha = 0.2; d = 2$			$\alpha = 0.3; d = 2$			$\alpha = 0.1; d = 3$			$\alpha = 0.2; d = 2$			$\alpha = 0.3; d = 2$		
Observed ratio (ρ)	0.5	0.7	1.0	0.5	0.7	1.0	0.5	0.7	1.0	0.5	0.7	1.0	0.5	0.7	1.0	0.5	0.7	1.0
iBAT [29]	0.126	0.157	0.199	0.161	0.213	0.304	0.195	0.278	0.331	0.154	0.233	0.409	0.217	0.313	0.504	0.254	0.326	0.585
SAE [17]	0.234	0.316	0.331	0.227	0.589	0.654	0.577	0.831	0.893	0.514	0.658	0.836	0.579	0.771	0.913	0.638	0.891	0.918
VSAE [10]	0.255	0.293	0.306	0.475	0.611	0.649	0.584	0.823	0.871	0.513	0.676	0.831	0.559	0.782	0.907	0.632	0.899	0.924
TPRRO [35]	0.278	0.289	0.292	0.285	0.297	0.303	0.312	0.324	0.322	0.517	0.518	0.553	0.524	0.539	0.551	0.519	0.537	0.543
GM-VSAE [15]	0.305	0.358	0.374	0.509	0.667	0.722	0.606	0.853	0.910	0.522	0.687	0.858	0.573	0.798	0.919	0.654	0.914	0.947
DeepTEA [7]	0.347	0.366	0.408	0.496	0.685	0.736	0.588	0.863	0.921	0.531	0.693	0.875	0.582	0.812	0.931	0.664	0.919	0.935
MST-OATD w/o T	0.372	0.464	0.507	0.535	0.720	0.808	0.609	0.880	0.947	0.523	0.694	0.872	0.598	0.804	0.926	0.681	0.929	0.953
MST-OATD w/o M	0.411	0.575	0.683	0.526	0.742	0.847	0.618	0.865	0.951	0.538	0.715	0.878	0.581	0.806	0.936	0.669	0.923	0.962
MST-OATD	**0.431**	**0.590**	**0.739**	**0.539**	**0.752**	**0.892**	**0.621**	**0.884**	**0.970**	**0.547**	**0.724**	**0.907**	**0.605**	**0.821**	**0.949**	**0.688**	**0.933**	**0.969**

- **MST-OATD w/o T**: This variant only uses spatial information for training and detects spatial anomalous trajectories.
- **MST-OATD w/o M**: This variant uses the single-scale inference, not multi-scale inference, on both the spatial and temporal dimensions.

MST-OATD outperforms the two variants, confirming the effectiveness of multi-scale inference and temporal embedding. We find that temporal embedding has a substantially larger impact on detection performance than has multi-scale inference in general. This suggests that the combination of spatial and temporal embedding is most critical to anomalous trajectory detection. Furthermore, the improvement due to using spatial and temporal embeddings is more significant when the anomaly distance is short or the anomaly fraction is small. This indicates that the temporal anomalies of a trajectory are more pronounced than the spatial anomalies. Multi-scale inference yields almost the same improvements for each set of experiments on both Porto and Chengdu.

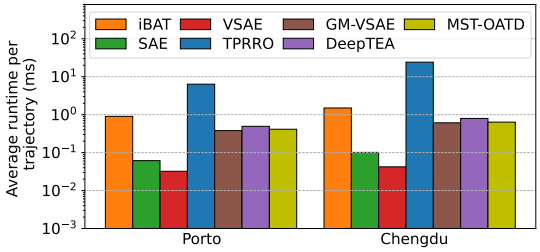


Figure 5: Detection time comparison.

7.5 Efficiency Evaluation

Figure 5 shows the average run time per trajectory. The efficiency tests are performed on a single NVIDIA RTX 3090 GPU. In the efficiency evaluation, the numbers of Gaussian components of GM-VSAE, DeepTEA, and MST-OATD are all set to 10. TPRRO and iBAT are the most time consuming methods as they extract normal routes first and then compare a trajectory with these routes. VSAE is the fastest deep learning method because it does not need to encode trajectories in a latent space as does SAE. Due to the parallel spatial and temporal generation, the detection speeds of MST-OATD and GM-VSAE are basically the same. However, due to the use of Gaussian mixture distributions, the detection time is proportional to the number of Gaussian components, making these two methods slower than VSAE and SAE. The detection speed of DeepTEA is lower than those of the other learning-based methods as it needs to capture traffic patterns before generating trajectories. It is worth noting that MST-OATD only takes 0.413 ms to detect a trajectory in Porto and 0.636 ms in Chengdu, which is sufficient for real-time detection.

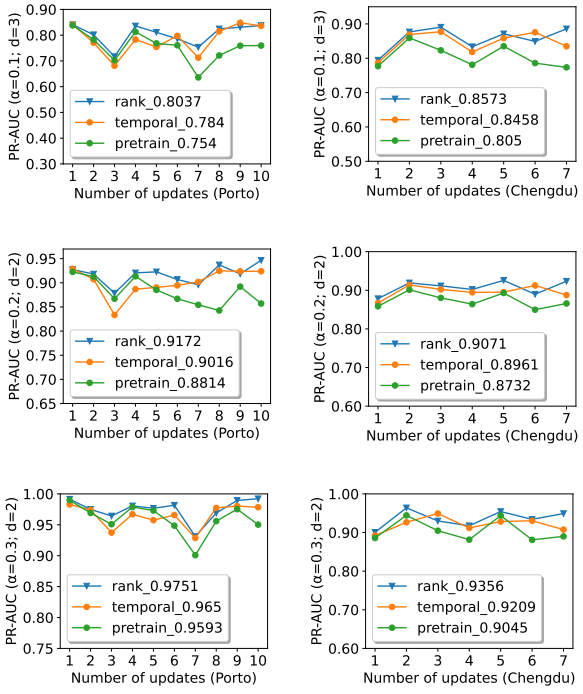


Figure 6: PR-AUC comparison of updatable training.

7.6 Updatable Training Evaluation

We further evaluate the effectiveness of our ranking-based method when performing updatable training. Figure 6 shows the PR-AUC

when detecting anomalous trajectories on Porto and Chengdu. We compare ranking-based approach with two other methods: temporal update and a pretrained model, denoted as temporal and pretrain in the figure. Temporal update replaces the oldest trajectories in the training set when new trajectories arrive, and the pretrained model keeps the weight of the model unchanged. We also report the average PR-AUC for the three methods in Figure 6. We can see that our proposed ranking-based method outperforms the pretrained model and the temporal update strategy. With the passage of time, there is some degradation in the effectiveness of the pre-trained model, as the characteristics of trajectories change over time. However, the performance of training with temporal updating and ranking-based updating does not show a significant decline.

8 CONCLUSION

We propose MST-OATD, a multi-scale model for online anomalous spatial trajectory detection. MST-OATD utilizes both spatial and temporal embeddings, and a multi-scale inference strategy is designed to improve the performance of the trajectory representation. MST-OATD can detect both spatial and temporal anomalies and supports online detection. We also introduce a learned ranking model to update the trajectory training set based on the distribution of incoming trajectories. We report on extensive experiments on two real-world, large-scale taxi trajectory datasets. The results offer evidence that each component of MST-OATD is effective and efficient and show that MST-OATD is capable of outperforming six baseline methods. The updatable training also shows that our proposed ranking-based update strategy is capable of state-of-the-art performance. In future research, it is of interest to extend our model to support other trajectory data mining tasks.

ACKNOWLEDGMENT

This work was supported by the NSFC (No. U22B2037, U2001212, U21B2046, 62032001, and 61932004).

REFERENCES

[1] Rudolf Bayer and Edward M. McCreight. 1972. Organization and Maintenance of Large Ordered Indices. *Acta Informatica* 1 (1972), 173–189.
[2] Chao Chen, Daqing Zhang, Pablo Samuel Castro, Nan Li, Lin Sun, Shijian Li, and Zonghui Wang. 2013. iBOAT: Isolation-Based Online Anomalous Trajectory Detection. *IEEE Trans. Intell. Transp. Syst.* 14, 2 (2013), 806–818.
[3] Bin Cheng, Shiyou Qian, Jian Cao, Guangtao Xue, Jiadi Yu, Yanmin Zhu, Minglu Li, and Tao Zhang. 2019. STL: Online Detection of Taxi Trajectory Anomaly Based on Spatial-Temporal Laws. In *DASFAA (Lecture Notes in Computer Science, Vol. 11447)*. Springer, 764–779.
[4] Jialin Ding, Umar Farooq Minhas, Jia Yu, Chi Wang, Jaeyoung Do, Yinan Li, Hantian Zhang, Badrish Chandramouli, Johannes Gehrke, Donald Kossmann, David B. Lomet, and Tim Kraska. 2020. ALEX: An Updatable Adaptive Learned Index. In *SIGMOD*. ACM, 969–984.
[5] Qiang Gao, Xiaohan Wang, Chaoran Liu, Goce Trajcevski, Li Huang, and Fan Zhou. 2023. Open Anomalous Trajectory Recognition via Probabilistic Metric Learning. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI 2023, 19th-25th August 2023, Macao, SAR, China*. ijcai.org, 2095–2103.
[6] Yong Ge, Hui Xiong, Chuanren Liu, and Zhi-Hua Zhou. 2011. A Taxi Driving Fraud Detection System. In *ICDM*. IEEE Computer Society, 181–190.
[7] Xiaolin Han, Reynold Cheng, Chenhao Ma, and Tobias Grubenmann. 2022. DeepTEA: Effective and Efficient Online Time-dependent Trajectory Outlier Detection. *Proc. VLDB Endow.* 15, 7 (2022), 1493–1505.
[8] Seyed Mehran Kazemi, Rishab Goel, Sepehr Eghbali, Janahan Ramanan, Jaspreet Sahota, Sanjay Thakur, Stella Wu, Cathal Smyth, Pascal Poupart, and Marcus A. Brubaker. 2019. Time2Vec: Learning a Vector Representation of Time. *CoRR* abs/1907.05321 (2019).
[9] Seokyeon Kim, Seongmin Jeong, Insoo Woo, Yun Jang, Ross Maciejewski, and David S. Ebert. 2018. Data Flow Analysis and Visualization for Spatiotemporal Statistical Data without Trajectory Information. *IEEE Trans. Vis. Comput. Graph.* 24, 3 (2018), 1287–1300.
[10] Diederik P. Kingma and Max Welling. 2014. Auto-Encoding Variational Bayes. In *ICLR*.
[11] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *ICLR*. OpenReview.net.
[12] Tim Kraska, Alex Beutel, Ed H. Chi, Jeffrey Dean, and Neoklis Polyzotis. 2018. The Case for Learned Index Structures. In *SIGMOD*. ACM, 489–504.
[13] Jae-Gil Lee, Jiawei Han, and Xiaolei Li. 2008. Trajectory Outlier Detection: A Partition-and-Detect Framework. In *ICDE*. IEEE Computer Society, 140–149.
[14] Xiucheng Li, Kaiqi Zhao, Gao Cong, Christian S. Jensen, and Wei Wei. 2018. Deep Representation Learning for Trajectory Similarity Computation. In *ICDE*. IEEE Computer Society, 617–628.
[15] Yiding Liu, Kaiqi Zhao, Gao Cong, and Zhifeng Bao. 2020. Online Anomalous Trajectory Detection with Deep Generative Sequence Modeling. In *ICDE*. IEEE, 949–960.
[16] Zhongjian Lv, Jiajie Xu, Pengpeng Zhao, Guanfeng Liu, Lei Zhao, and Xiaofang Zhou. 2017. Outlier Trajectory Detection: A Trajectory Analytics Based Approach. In *DASFAA*, Vol. 10177. Springer, 231–246.
[17] Pankaj Malhotra, Anusha Ramakrishnan, Gaurangi Anand, Lovekesh Vig, Puneet Agarwal, and Gautam Shroff. 2016. LSTM-based Encoder-Decoder for Multi-sensor Anomaly Detection. *CoRR* abs/1607.00148 (2016).
[18] Yue Ning, Liang Zhao, Feng Chen, Chang-Tien Lu, and Huzefa Rangwala. 2019. Spatio-temporal Event Forecasting and Precursor Identification. In *KDD*. ACM, 3237–3238.
[19] Xuan Rao, Hao Wang, Liang Zhang, Jing Li, Shuo Shang, and Peng Han. 2022. FOGS: First-Order Gradient Supervision with Learning-based Graph for Traffic Flow Forecasting. In *IJCAI*. ijcai.org, 3926–3932.
[20] Li Song, Ruijia Wang, Ding Xiao, Xiaotian Han, Yanan Cai, and Chuan Shi. 2018. Anomalous Trajectory Detection Using Recurrent Neural Network. In *ADMA*, Vol. 11323. Springer, 263–277.
[21] Yueyang Su, Di Yao, Xiaolei Zhou, Yuxuan Zhang, Yunxia Fan, Lu Bai, and Jingping Bi. 2023. TripSafe: Retrieving Safety-related Abnormal Trips in Real-time with Trajectory Data. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2023, Taipei, Taiwan, July 23-27, 2023*. ACM, 2446–2450.
[22] Yao Tian, Tingyun Yan, Xi Zhao, Kai Huang, and Xiaofang Zhou. 2023. A Learned Index for Exact Similarity Search in Metric Spaces. *IEEE Trans. Knowl. Data Eng.* 35, 8 (2023), 7624–7638.
[23] Sheng Wang, Zhifeng Bao, J. Shane Culpepper, and Gao Cong. 2022. A Survey on Trajectory Data Management, Analytics, and Learning. *ACM Comput. Surv.* 54, 2 (2022), 39:1–39:36.
[24] Hao Wu, Weiwei Sun, and Baihua Zheng. 2017. A Fast Trajectory Outlier Detection Approach via Driving Behavior Modeling. In *CIKM*. ACM, 837–846.
[25] Dongbo Xi, Fuzhen Zhuang, Yanchi Liu, Jingjing Gu, Hui Xiong, and Qing He. 2019. Modelling of Bi-Directional Spatio-Temporal Dependence and Users' Dynamic Preferences for Missing POI Check-In Identification. In *AAAI*. AAAI Press, 5458–5465.
[26] Chuang Yang, Zhiwen Zhang, Zipei Fan, Renhe Jiang, Quanjun Chen, Xuan Song, and Ryosuke Shibasaki. 2023. EpiMob: Interactive Visual Analytics of Citywide Human Mobility Restrictions for Epidemic Control. *IEEE Trans. Vis. Comput. Graph.* 29, 8 (2023), 3586–3601.
[27] Zhuoning Yuan, Xun Zhou, and Tianbao Yang. 2018. Hetero-ConvLSTM: A Deep Learning Approach to Traffic Accident Prediction on Heterogeneous Spatio-Temporal Data. In *KDD*. ACM, 984–992.
[28] Dongxiang Zhang, Zhihao Chang, Sai Wu, Ye Yuan, Kian-Lee Tan, and Gang Chen. 2022. Continuous Trajectory Similarity Search for Online Outlier Detection. *IEEE Trans. Knowl. Data Eng.* 34, 10 (2022), 4690–4704.
[29] Daqing Zhang, Nan Li, Zhi-Hua Zhou, Chao Chen, Lin Sun, and Shijian Li. 2011. iBAT: detecting anomalous taxi trajectories from GPS traces. In *UbiComp*. ACM, 99–108.
[30] Huichu Zhang, Chang Liu, Weinan Zhang, Guanjie Zheng, and Yong Yu. 2020. GeneraLight: Improving Environment Generalization of Traffic Signal Control via Meta Reinforcement Learning. In *CIKM*. ACM, 1783–1792.
[31] Kai Zheng, Yu Zheng, Nicholas Jing Yuan, and Shuo Shang. 2013. On discovery of gathering patterns from trajectories. In *ICDE*. IEEE Computer Society, 242–253.
[32] Kai Zheng, Yu Zheng, Nicholas Jing Yuan, Shuo Shang, and Xiaofang Zhou. 2014. Online Discovery of Gathering Patterns over Trajectories. *IEEE Trans. Knowl. Data Eng.* 26, 8 (2014), 1974–1988.
[33] Yu Zheng. 2015. Trajectory Data Mining: An Overview. *ACM Trans. Intell. Syst. Technol.* 6, 3 (2015), 29:1–29:41.
[34] Jie Zhu, Wei Jiang, An Liu, Guanfeng Liu, and Lei Zhao. 2015. Time-Dependent Popular Routes Based Trajectory Outlier Detection. In *WISE*, Vol. 9418. Springer, 16–30.
[35] Jie Zhu, Wei Jiang, An Liu, Guanfeng Liu, and Lei Zhao. 2017. Effective and efficient trajectory outlier detection based on time-dependent popular route. *World Wide Web* 20, 1 (2017), 111–134.

A NOTATION TABLE

For reference, Table 3 summarizes the key notations and their descriptions in this paper.

Table 3: Notation Table

Notation	Description
p	Trajectory point
T	Trajectory
$T^{(s)}$ / $T^{(t)}$	Spatial/temporal trajectory
$\mathbf{e}$	Trajectory embedding
$\mathbf{e}^{(s)}$ / $\mathbf{e}^{(t)}$	Spatial/temporal trajectory embedding
c	A route type
$rank_c(T)$	Trajectory rank of T in type c
k	A trajectory scale
$\mathbf{e}_k$	Trajectory embedding at scale k
r	Latent vector
$r^{(s)}$ / $r^{(t)}$	Spatial/temporal latent vector
r_T	Latent vector of trajectory T
$q_\theta(\cdot)$	Inference network
$p_\phi(\cdot)$	Distribution of route types
$p_\gamma(\cdot)$	Generative network
μ, σ	Parameters of Gaussian distribution
$f(\cdot)$	Fully connected layers
$g(\cdot)$	RNN modules
h, m	Hidden states of RNNs
N	Number of trajectories

B ADDITIONAL OPTIMIZATION DETAILS

The marginal log-likelihood is optimized by maximizing the lower bound on the sum of marginal likelihoods $\mathcal{L}(\theta, \phi, \gamma; T^{(s)})$ and $\mathcal{L}(\theta, \phi, \gamma; T^{(t)})$, as specified in Equation 23:

$$\begin{aligned}
&\log p_\gamma(T^{(s)}) p_\gamma(T^{(t)}) \geq \mathcal{L}(\theta, \gamma, \phi; T^{(s)}) + \mathcal{L}(\theta, \gamma, \phi; T^{(t)}) \\
&= \mathbb{E}_{q_\theta(r^{(s)}, c^{(s)}|T^{(s)})}[\frac{\log p_{\gamma,\phi}(T^{(s)}, r^{(s)}, c^{(s)})}{q_\theta(r^{(s)}, c^{(s)}|T^{(s)})}] \\
&+ \mathbb{E}_{q_\theta(r^{(t)}, c^{(t)}|T^{(t)})}[\frac{\log p_{\gamma,\phi}(T^{(t)}, r^{(t)}, c^{(t)})}{q_\theta(r^{(t)}, c^{(t)}|T^{(t)})}]
\end{aligned} \tag{23}$$

The lower bound can be converted to the form of Equation 24:

$$\begin{aligned}
&\mathbb{E}_{q_\theta(r,c|T)}[\log \frac{p_{\gamma,\phi}(T, r, c)}{q_\theta(r, c|T)}] = \mathbb{E}_{q_\theta(r|T)} \log p_\gamma(T|r) \\
&- \mathbb{E}_{q_\theta(c|T)}[D_{KL}(q_\theta(c|T)||p_\phi(c))] - \mathbb{E}_{q_\theta(c|T)}[D_{KL}(q_\theta(r|T)||p_\phi(r|c))],
\end{aligned} \tag{24}$$

where $\mathbb{E}_{q_\theta} \log p_\gamma(T|r)$ denotes the reconstruction error. $D_{KL}(\cdot||\cdot)$ is the KL divergence between two distributions. The first term is the reconstruction loss. It reflects the loss between the generated and the real trajectories. The second is the category loss, which is the loss between the distribution of trajectory types and the multinomial distribution. The third is the Gaussian loss, which is the loss between the latent vector and a specific Gaussian distribution.

C DETAILS ON EVOLVING DATASET UPDATE

Algorithm 1 The Evolving Dataset Update Algorithm

Input: Training set $U = \{U_1, U_2, ..., U_C\}$;
A new collected trajectory T
Output: Updated training set $\tilde{U}$

1: Construct a rank set $\mathcal{R} \leftarrow \emptyset$;
2: **for** every route type c **do**
3: Calculate the probability of T following type c $p(c|T)$;
4: Obtain $rank_c(T)$ based on the learned ranking model;
5: $\mathcal{R} \leftarrow \mathcal{R} \cup \{rank_c(T)\}$;
6: **end for**
7: Obtain the highest ranked category: $c_{max} \leftarrow \arg\max_c \mathcal{R}$;
8: $U_{c_{max}} \leftarrow U_{c_{max}} \cup \{T\}$;
9: Obtain the lowest ranked category: $c_{min} \leftarrow \arg\min_c \mathcal{R}$;
10: **while** $U_{c_{min}} = \emptyset$ **do**
11: $\mathcal{R} \leftarrow \mathcal{R} - \{rank_{c_{min}}(T)\}$;
12: $c_{min} \leftarrow \arg\min_c \mathcal{R}$;
13: **end while**
14: $U_{c_{min}} \leftarrow U_{c_{min}} - \{T_{oldest_c_{min}}\}$;
15: $\tilde{U} \leftarrow \{U_1, U_2, ..., U_C\}$;
16: **return** $\tilde{U}$;

The whole process of updating an evolving dataset is shown in Algorithm 1. The input is the original training set U and a newly collected trajectory T, and U can be divided into C categories, $U = \{U_1, U_2, ..., U_C\}$. The output is the updated training set $\tilde{U}$. First, for each type of route, we calculate the probability that trajectory T follows type c and use the probability to obtain the trajectory rank $rank_c(T)$ based on the learned ranking model, a set $\mathcal{R}$ is used to store all the ranks of trajectory T (Lines 1–6). After obtaining the ranks of each type, we calculate the highest ranked category c_{max} and add trajectory T to the subset $U_{c_{max}}$ (Lines 8–9). Using the same method, we calculate the lowest ranked category c_{min} and remove the oldest trajectory in category c_{min}, which denotes as $T_{oldest_c_{min}}$. During the update process, as we remove the oldest trajectory from the lowest ranked category set when a new trajectory becomes available, there may be cases where the lowest ranked category set, denoted as $U_{c_{min}}$, becomes empty. In such cases, we instead remove the oldest trajectory from the second-lowest ranked category, provided it is not empty, and so forth (Lines 9–14). Finally, we obtain the updated training set $\tilde{U}$ and return $\tilde{U}$ as the result (Lines 15–16).

D EXPERIMENTS

D.1 Implementation Details

We implement MST-OATD using Python 3.10.12 and Pytorch 2.0.1. Experiments are run on a machine with a 40-core Intel Xeon CPU, an NIVIDIA RTX 3090 GPU, and 128 GB of RAM. The AdamW optimizer, initialized with a learning rate of 0.002 and decaying by 0.9 after every 2 epochs, is used for model training. The batch size is set to 1600 for the Porto dataset and 300 for the Chengdu dataset. The embedding dimension is 128 and the hidden state dimension is

512 in our method, the same settings are applied to the baselines. We use 3 different scales for multi-scale inference, with scale sizes of 1, 2, and 4. For training on evolving datasets, the size of the training set is set to 80,000 for Porto and 200,000 for Chengdu. We repeat the experiments 5 times with different random seeds and report the average performance.

D.2 Experiments for longer trajectories on Porto dataset

To evaluate the effectiveness of our method in detecting longer trajectories, we clip trajectories to a maximum of 100 points in the Porto dataset and perform experiments. The preprocessed dataset contains 1,162,423 trajectories and 37,856,009 points. The performance comparison with GM-VSAE and DeepTEA is shown in Table 4. We have two observations: First, our proposed MST-OATD still outperforms the state-of-the-art methods, DeepTEA and GM-VSAE, when the maximum trajectory length is set to 100; Second, setting the maximum trajectory length to 50 and 100 maintains almost the same detection performance, indicating that our method is applicable to the detection of trajectory anomalies of different lengths.

Table 4: Performance for longer trajectories on Porto dataset

Parameters	$\alpha = 0.1; d = 3$			$\alpha = 0.2; d = 2$			$\alpha = 0.3; d = 2$		
Observed ratio	0.5	0.7	1.0	0.5	0.7	1.0	0.5	0.7	1.0
GM-VSAE	0.345	0.396	0.382	0.514	0.667	0.709	0.614	0.845	0.904
DeepTEA	0.374	0.404	0.413	0.506	0.679	0.729	0.596	0.848	0.921
MST-OATD	0.419	0.619	0.744	0.528	0.754	0.887	0.616	0.891	0.974